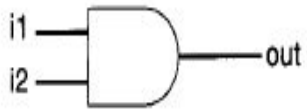




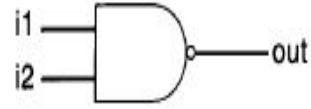
Verilog - predefined primitives

Go, change the world®

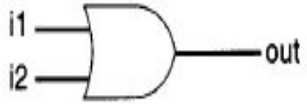
- Verilog includes **12 basic logic gates** as predefined primitives.
- **Four** of these primitive gates are of the **three-state type**.
- The other eight are the basic gates.
- They are declared with the lowercase keywords **and, nand, or, nor, xor, xnor, not, and buf**.
- Primitives such as **and** are n-input primitives, because they can have any number of scalar inputs (e.g., a three-input and primitive).
- The **buf** and **not** primitives are n-output primitives because a single input to a buf or not gate can drive multiple outputs.



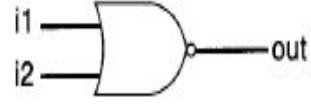
and



nand



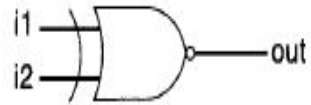
or



nor



xor



xnor

```
wire OUT, IN1, IN2;
```

```
//basic gate instantiations.
```

```
and al(OUT, IN1, IN2);
```

```
nand nal (OUT, IN1, IN2);
```

```
or orl(OUT, IN1, IN2);
```

```
nor nor1 (OUT, IN1, IN2);
```

```
xor xl (OUT, IN1, IN2);
```

```
xnor nxl (OUT, IN1, IN2);
```

```
//More than two inputs; 3 input nand gate
```

```
nand nal_3inp (OUT, IN1, IN2, IN3);
```

```
//gate instantiation without instance name
```

```
and (OUT, IN1, IN2); //legal gate  
instantiation
```



	and	i1			
		0	1	x	z
i2	0	0	0	0	0
	1	0	1	x	x
	x	0	x	x	x
	z	0	x	x	x

	nand	i1			
		0	1	x	z
i2	0	1	1	1	1
	1	1	0	x	x
	x	1	x	x	x
	z	1	x	x	x

	or	i1			
		0	1	x	z
i2	0	0	1	x	x
	1	1	1	1	1
	x	x	1	x	x
	z	x	1	x	x

	nor	i1			
		0	1	x	z
i2	0	1	0	x	x
	1	0	0	0	0
	x	x	0	x	x
	z	x	0	x	x



Buf/Not Gates

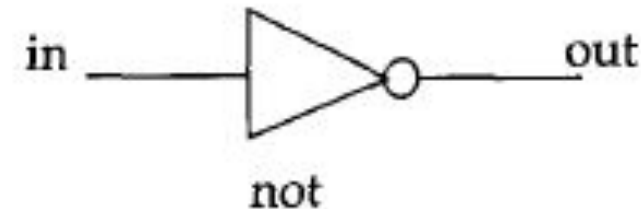
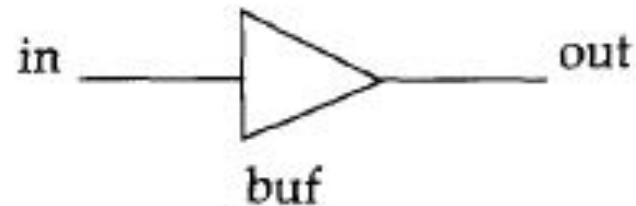
Buf/not gates have one scalar input and one or more scalar outputs.

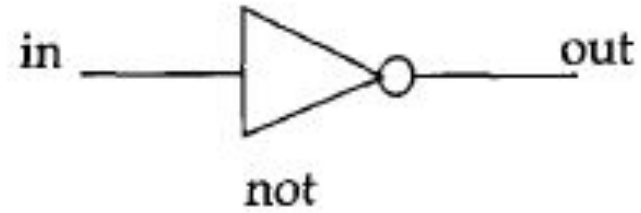
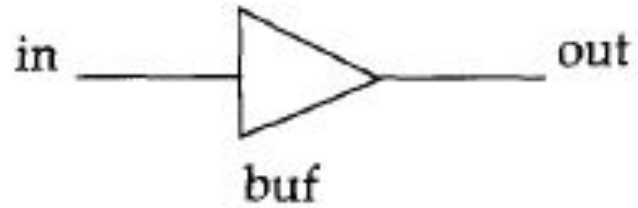
The last terminal in the port list is connected to the input.

Other terminals are connected to the outputs.

We will discuss gates that have one input and one output.

Two basic *buf/not* gate primitives are provided in Verilog.





```
// basic gate instantiations.
```

```
buf b1(OUT1, IN);
```

```
not n1(OUT1, IN);
```

```
// More than two outputs
```

```
buf b1_2out(OUT1, OUT2, IN);
```

```
// gate instantiation without instance name
```

```
not (OUT1, IN); // legal gate instantiation
```

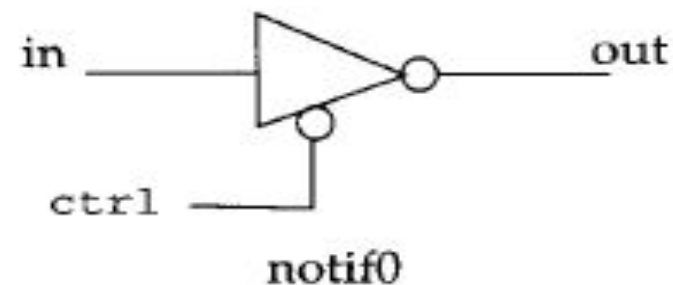
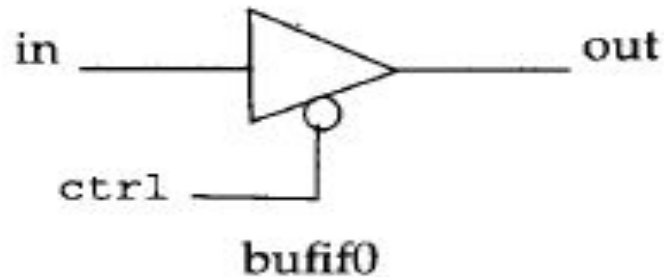
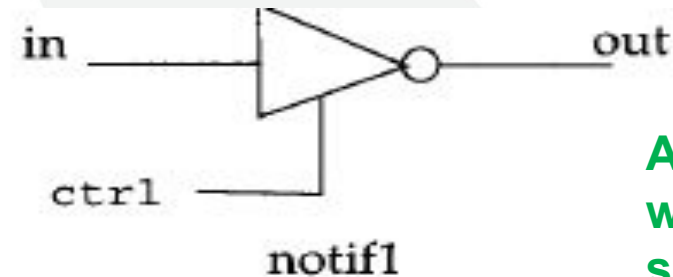
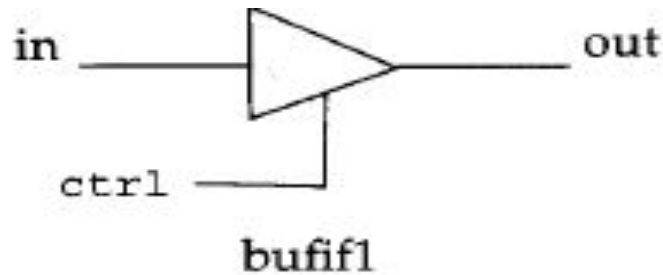
Truth Tables for Buf/Not gates

buf	in	out
	0	0
	1	1
	x	x
	z	x

not	in	out
	0	1
	1	0
	x	x
	z	x

Bufif/Notif Gates

- Gates with an additional control signal on **buf** and **not** gates are also available.
- These gates propagate only if their control signal is asserted.
- They propagate **z** if their control signal is deasserted



Application: Used in bus systems where multiple devices share the same line, but only one should drive it at a time.



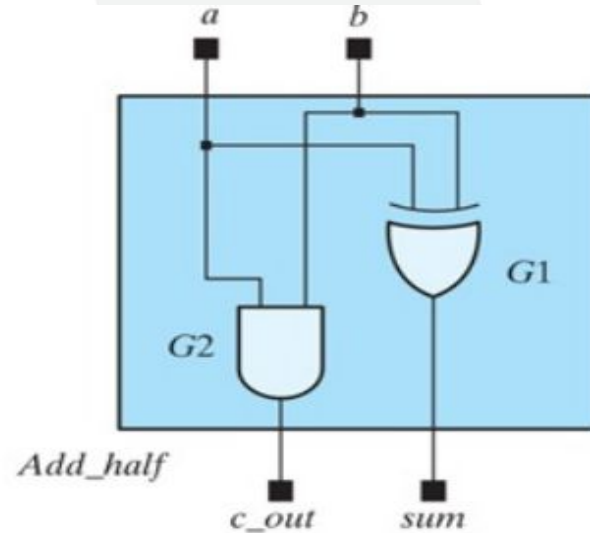
		ctrl			
bufif1		0	1	x	z
in	0	z	0	L	L
	1	z	1	H	H
	x	z	x	x	x
	z	z	x	x	x

		ctrl			
notif1		0	1	x	z
in	0	z	1	H	H
	1	z	0	L	L
	x	z	x	x	x
	z	z	x	x	x

		ctrl			
bufif0		0	1	x	z
in	0	0	z	L	L
	1	1	z	H	H
	x	x	z	x	x
	z	x	z	x	x

		ctrl			
notif0		0	1	x	z
in	0	1	z	H	H
	1	0	z	L	L
	x	x	z	x	x
	z	x	z	x	x

Structural design: Half adder using Gates

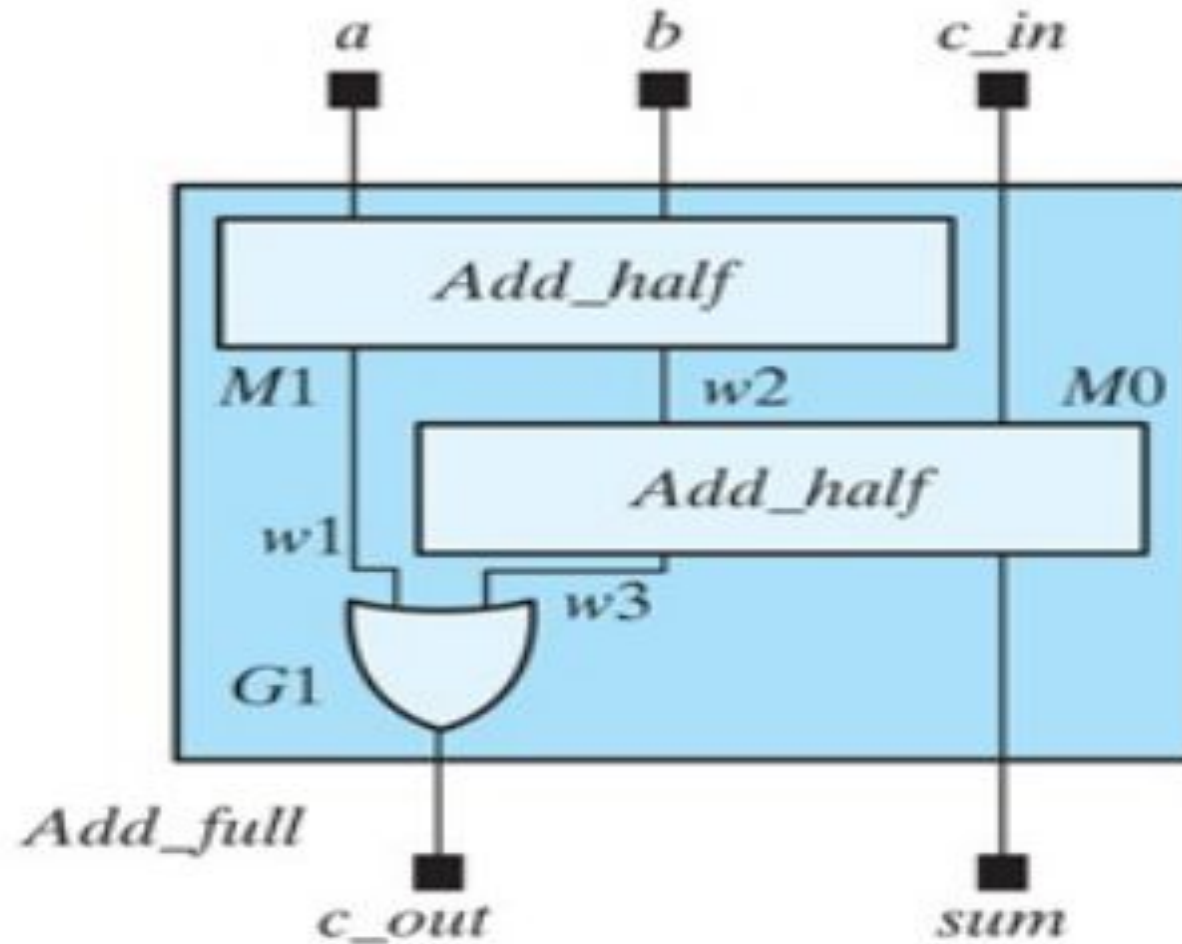


```
module Add_half (input a, b, output c_out, sum),  
    xor G1(sum, a, b);           // Gate instance names are option  
    and G2(c_out, a, b);  
endmodule
```



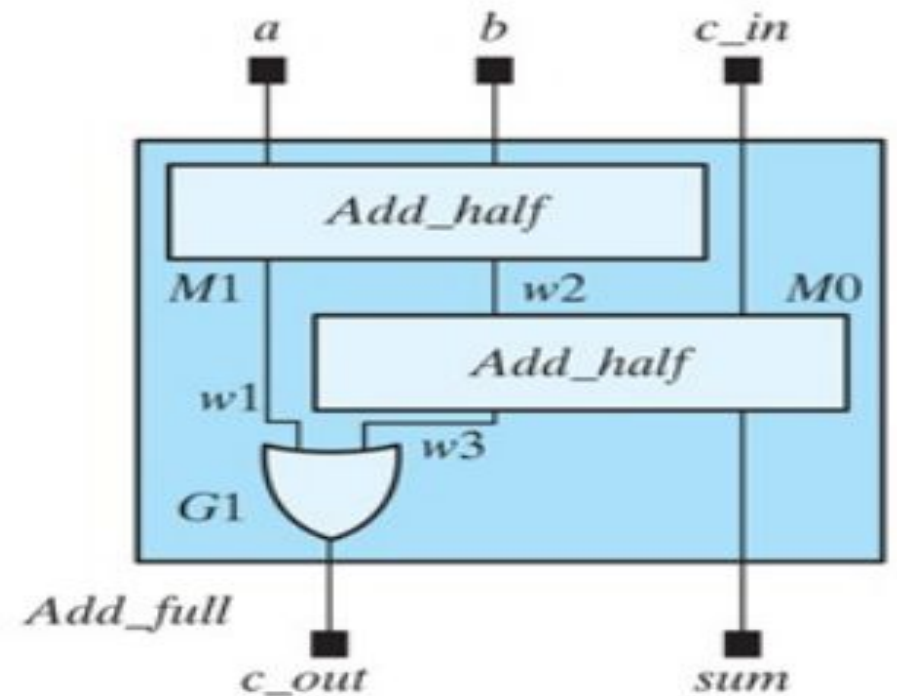
RV College of
Engineering®

Structural design: Full Adder using two half adder





Structural design: Full Adder using two half adder

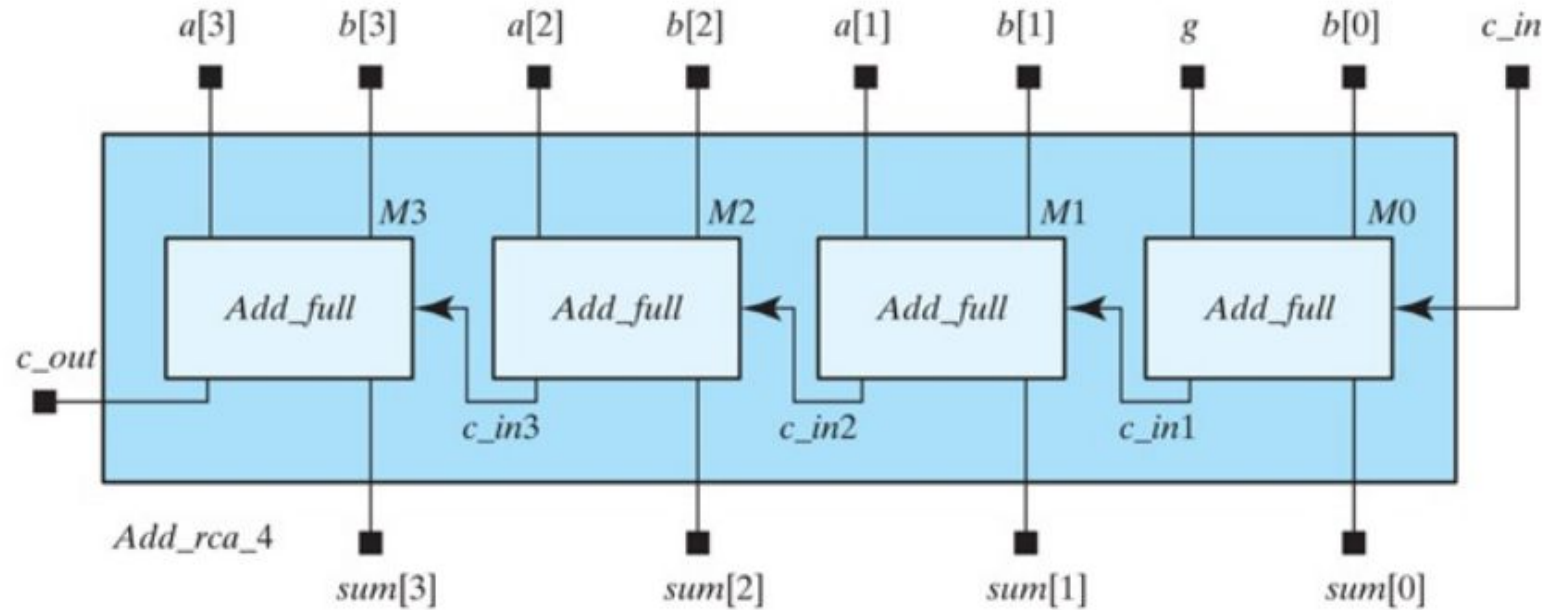


```
module Add_full (input a, b, c_in, output c_out, sum);    // see  
Fig.  
4.8
```

```
    wire w1, w2, w3;                                // w1 is c_out; w2 is sum  
    Add_half M1 (a, b, w1, w2);  
    Add_half M0 (w2, c_in, w3, sum);  
    or (c_out, w1, w3);  
endmodule
```



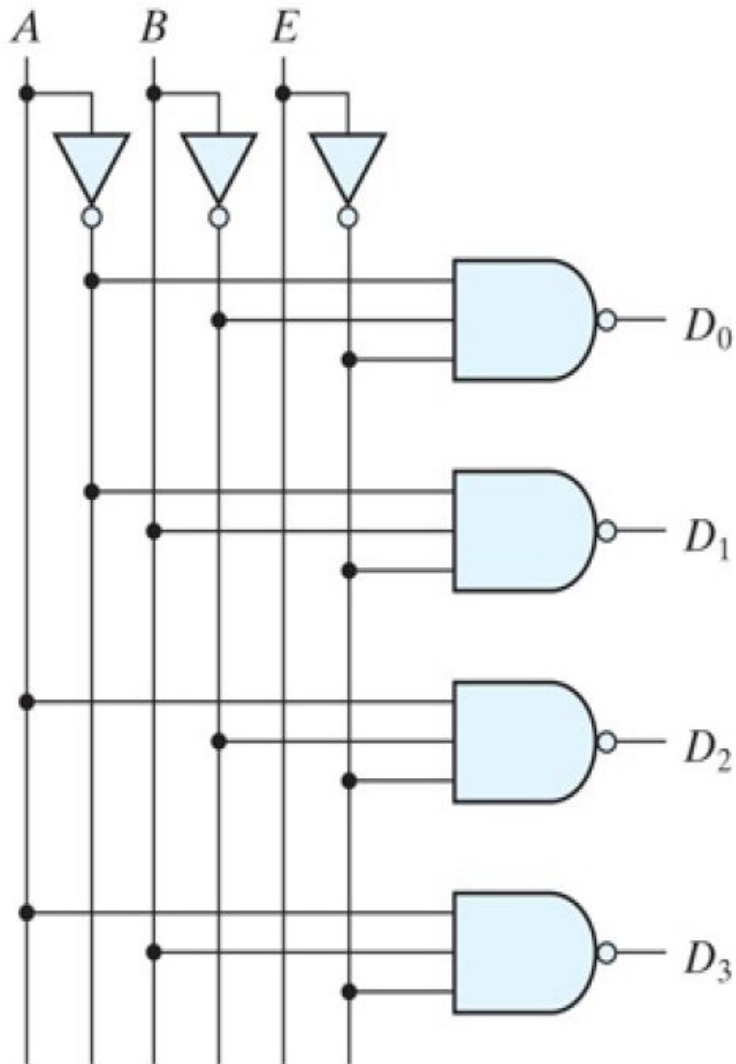
Structural design: 4 bit Adder using 4 full adders



```
module Add_rca_4 (input [3:0] a, b, input c_in output c_out, ou
    wire c_in1, c_in3, c_in4;           // Intermediate carries
    Add_full M0 (a[0], b[0], c_in, c_in1, sum[0]);
    Add_full M1 (a[1], b[1], c_in1, c_in2, sum[1]);
    Add_full M2 (a[2], b[2], c_in2, c_in3, sum[2]);
    Add_full M3 (a[3], b[3], c_in3, c_out, sum[3]);
endmodule
```

Structural design- 2:4 Decoder

Go, change the world®



<i>E</i>	<i>A</i>	<i>B</i>				
			<i>D</i> ₀	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0



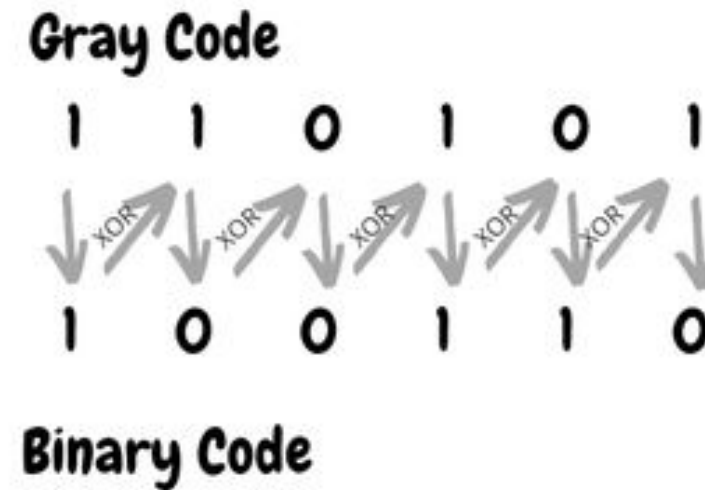
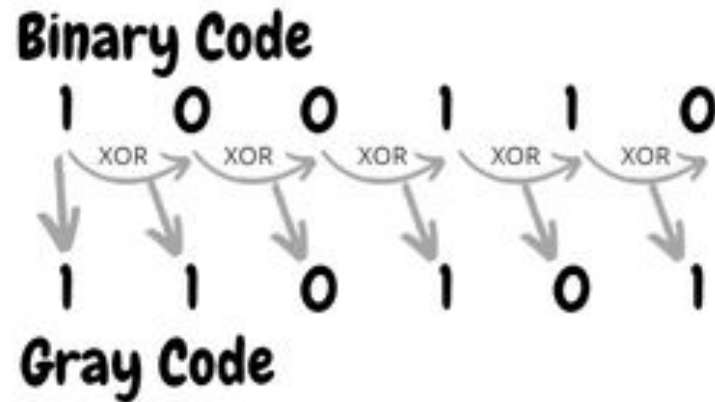
Structural design- 2:4 Decoder

Go, change the world®

```
// Gate-level description of two-to-four-line decoder
// Refer to Fig.4.19 with symbol E replaced by enable, for clarity
module decoder_2x4_gates (D, A, B, enable);
    output [0: 3] D;
    input          A, B;
    input          enable;
    wire           A_not, B_not, enable_not;
    not
        G1 (A_not, A),           // Comma-separated list of primitives
        G2 (B_not, B),
        G3 (enable_not, enable);
    nand
        G4 (D[0], A_not, B_not, enable_not),
        G5 (D[1], A_not, B, enable_not),
        G6 (D[2], A, B_not, enable_not),
        G7 (D[3], A, B, enable_not);
endmodule
```

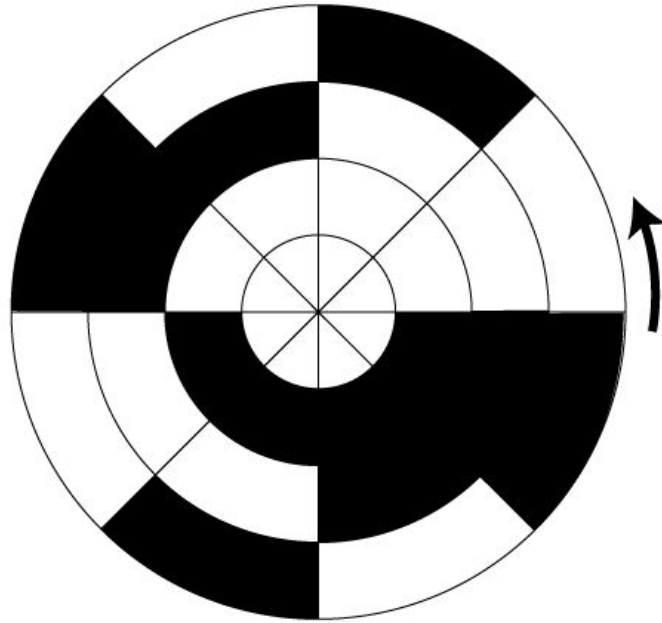
Note that the keywords **not** and **nand** are written only once and do not have to be repeated for each instance of the **nand** gate, but commas must be inserted at the end of each instantiation of the gates in the series, except for the last statement, which must be terminated by a semicolon.

Design a 5-bit Gray to Binary converter using gate level modelling





Encoder Gray Code



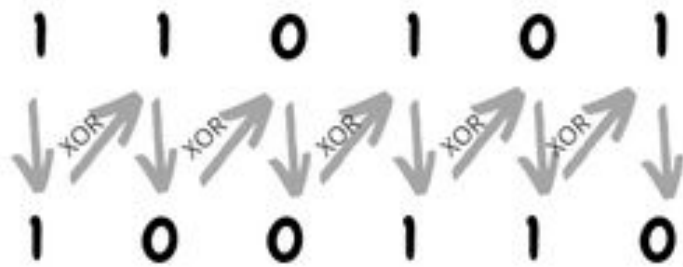
a) Gray Code

signal#1	signal#2	signal#3	Angle
0	0	0	0° to 45°
0	0	1	45° to 90°
0	1	0	90° to 135°
0	1	1	135° to 180°
1	0	0	180° to 225°
1	0	1	225° to 270°
1	1	0	270° to 315°
1	1	1	315° to 360°

b) Signal Outputs

Design a 5-bit Gray to Binary converter using gate level modelling

Gray Code



Binary Code

```
module gray_to_binary (  
    input [4:0] gray, // 5-bit Gray code input  
    output [4:0] binary // 5-bit Binary code output  
);  
    assign binary[4]=gray[4];  
    xor(binary[3],binary[4],gray[3]);  
    xor(binary[2],binary[3],gray[2]);  
    xor(binary[1],binary[2],gray[1]);  
    xor(binary[0],binary[1],gray[0]);  
endmodule
```



Dataflow model

Go, change the world®

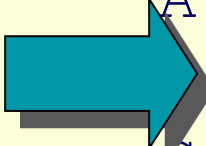
- Dataflow models describe **combinational circuits** by their *function* rather than by their gate structure.
- A common form of dataflow modeling of combinational logic uses **concurrent signal assignment statements** and built-in language operators to express how signals are assigned values.



Logical Operators

Go, change the world®

- `& &` → logical AND
- `| |` → logical OR
- `!` → logical NOT
- Operands evaluated to ONE bit value: 0, 1 or x
- Result is ONE bit value: 0, 1 or x

<code>A = 6;</code>		<code>A && B → 1 && 0 → 0</code>
<code>B = 0;</code>		<code> !B → 1 1 → 1</code>
<code>C = x;</code>		<code>C B → x 0 → x</code>



Bitwise Operators (i) & Reduction operator

& → bitwise AND
| → bitwise OR
~ → bitwise NOT
^ → bitwise XOR
~^ or ^~ → bitwise XNOR
Operation on bit by bit basis

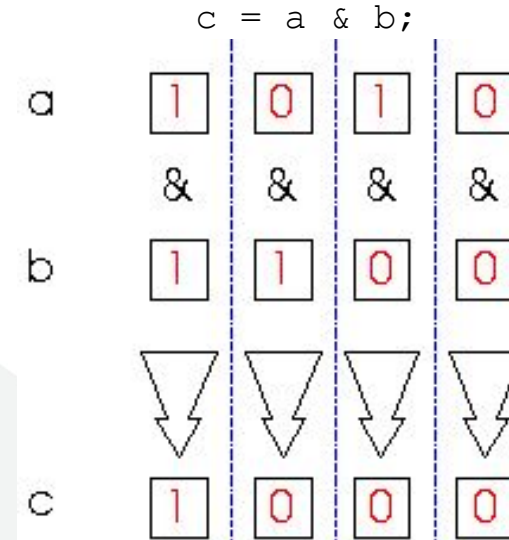
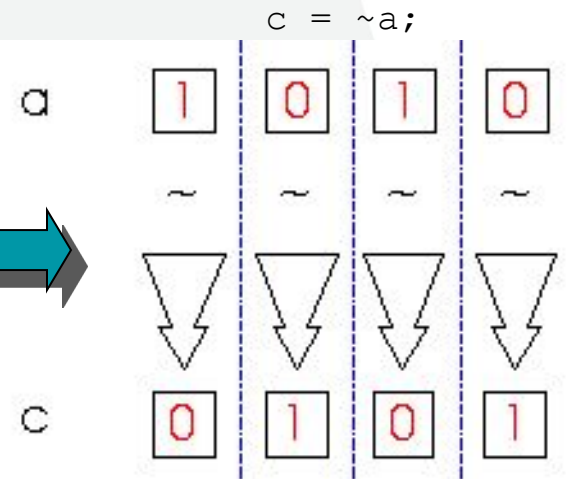
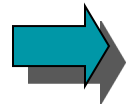
- & → AND
- | → OR
- ^ → XOR
- ~& → NAND
- ~| → NOR
- ~^ or ^~ → XNOR
- One multi-bit operand → One single-bit result

```
a = 4'b1001;  
c = |a; // c = 1|0|0|1 = 1  
Y= &A; // Y= 1&0&0&1 = 0
```

Bitwise Operators (ii)

Go, change the world®

- `a = 4'b1010;`
`b = 4'b1100;`

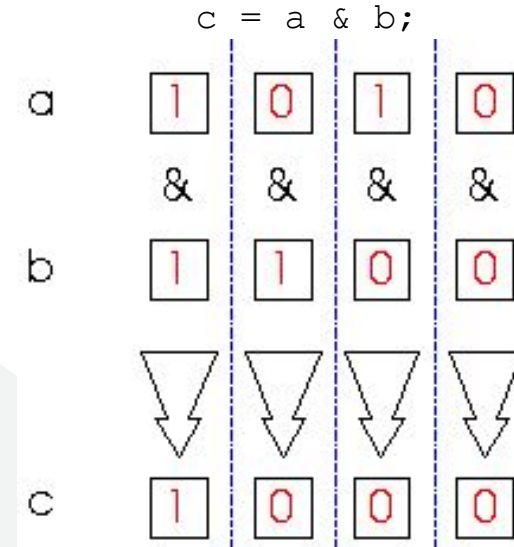
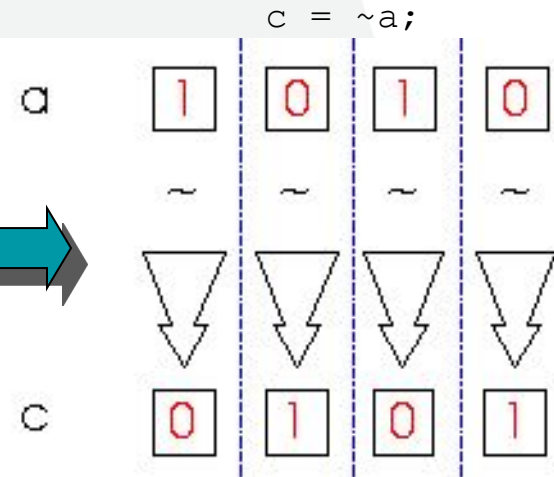
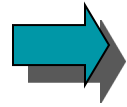


`a = 4'b1010;`
`b = 2'b11;`
`c = a ^ b; .....?`

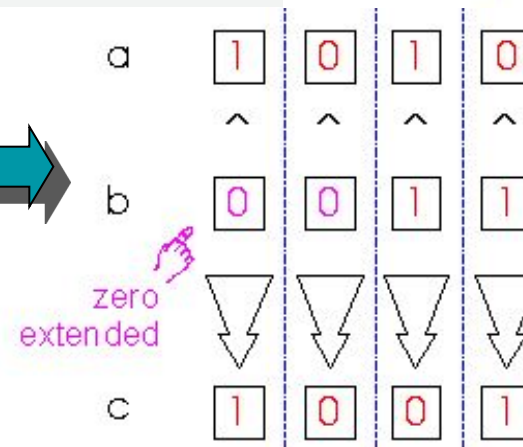
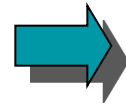
Bitwise Operators (ii)

Go, change the world®

- $a = 4'b1010;$
 $b = 4'b1100;$



$a = 4'b1010;$
 $b = 2'b11;$ $c = a \wedge b; ? \dots$





```
module reduction_example;

reg [3:0] a;

initial begin
    a = 4'b1011;

    $display("a      = %b", a);
    $display("&a     = %b (AND reduction)", &a);
    $display("~&a    = %b (NAND reduction)", ~&a);
    $display("|a     = %b (OR reduction)", |a);
    $display("~|a    = %b (NOR reduction)", ~|a);
    $display("^a     = %b (XOR reduction)", ^a);
    $display("~^a    = %b (XNOR reduction)", ~^a);

end

endmodule
```



```
module reduction_example;  
  reg [3:0] a;  
  initial begin  
    a = 4'b1011;  
    $display("a      = %b", a);  
    $display("&a     = %b (AND reduction)", &a);  
    $display("~&a    = %b (NAND reduction)", ~&a);  
    $display("|a     = %b (OR reduction)", |a);  
    $display("~|a    = %b (NOR reduction)", ~|a);  
    $display("^a     = %b (XOR reduction)", ^a);  
    $display("~^a    = %b (XNOR reduction)", ~^a);  
  end  
endmodule
```

$a = 1011$

$\&a = 0$

$\sim\&a = 1$

$|a = 1$

$\sim|a = 0$

$\wedge a = 1$

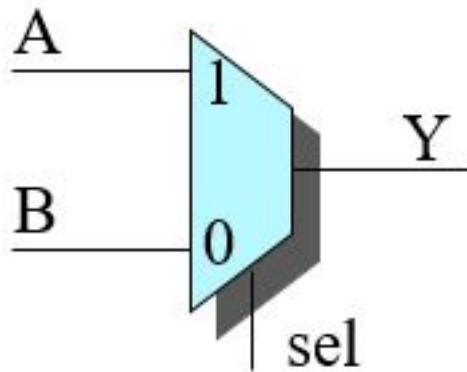
$\sim\wedge a = 0$

Why Do we need them ?

Conditional Operator

Go, change the world®

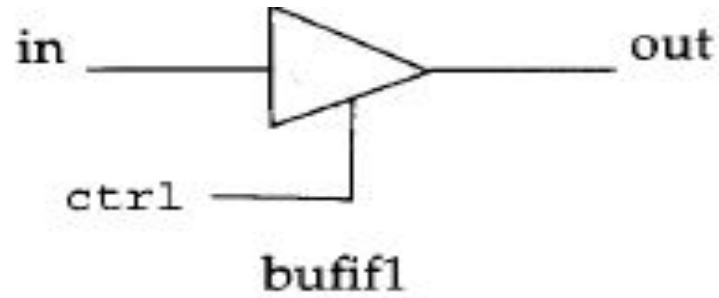
- **`cond_expr ? true_expr : false_expr`**
- 2-to-1 mux ..



`Y = (sel) ? A : B;`

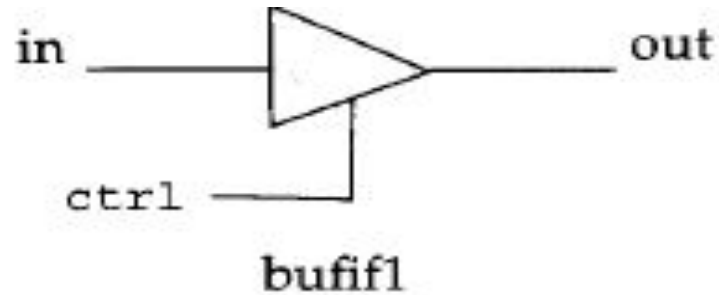


Write a Verilog code for 32 bit wide tri state buffer using data flow modelling



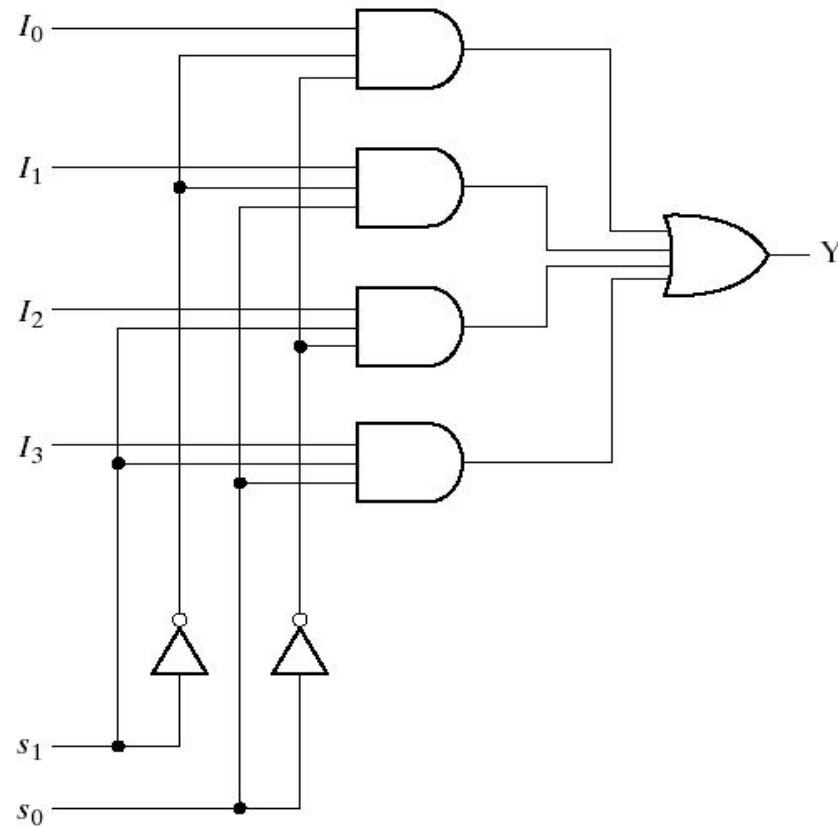


Write a Verilog code for 32 bit wide tri state buffer using data flow modelling



```
module tri_buf(out, in, ctrl);  
    output [31:0] out;  
    input [31:0] in; input ctrl;  
  
    assign out=ctrl? in: 32'bz;  
  
endmodule
```

4:1 MUX using conditional operator



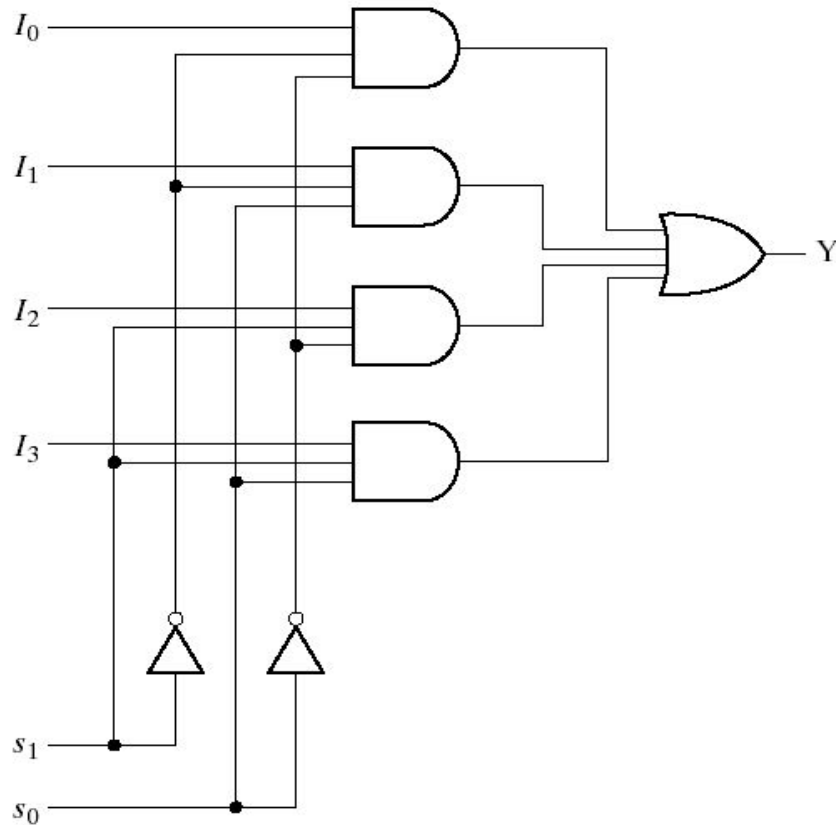
(a) Logic diagram

s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

4:1 MUX using conditional operator

Go, change the world®



(a) Logic diagram

s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

```
module mux4to1(output y, input i0, i1, i2, i3, s0, s1);
  assign y = s1 ? ( s0 ? i3:i2) : (s0 ? i1:i0);
  //nested conditional operator
endmodule
```

4X1 - [C:/Users/Chanchal/vlvAdO/Verilog/4X1/4X1.xpr] - Vivado 2019.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Ready

Default Layout

Flow Navigator

ELABORATED DESIGN - xc7vx485tffg1157-1

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Open Elaborated Design
 - Report Methodology
 - Report DRC
 - Report Noise
 - Schematic

SYNTHESIS

- Run Synthesis
- Open Synthesized Design

IMPLEMENTATION

- Run Implementation

Properties

Sources

Netlist

Project Summary x Schematic x mux.v x tb.v x Schematic (2) x

3 Cells 7 I/O Ports 9 Nets

The diagram illustrates a hierarchical 4-to-1 Multiplexer (MUX) structure implemented using three RTL_MUX blocks. The inputs are divided into two groups of two: inputs 'b' and 'a' for the first MUX (labeled 'out0_i_0'), and inputs 'd' and 'c' for the second MUX (labeled 'out0_i'). Each MUX has a select input 'S' and two data inputs 'I0' and 'I1'. The first MUX's output 'O' is connected to the 'I0' input of the third MUX (labeled 'out_i'). The second MUX's output 'O' is connected to the 'I1' input of the third MUX. The select input 'S' of the third MUX is connected to the output of a 2-to-1 MUX formed by inputs 's0' and 's1'. The final output of the third MUX is labeled 'out'.

out0_i_0

out0_i

out_i

out

RTL_MUX

RTL_MUX

RTL_MUX

Tcl Console Messages Log Reports Design Runs



Dataflow model

Go, change the world®

- Dataflow models describe **combinational circuits** by their *function* rather than by their gate structure.
- A common form of dataflow modeling of combinational logic uses **concurrent signal assignment statements** and built-in language operators to express how signals are assigned values.

Example

```
module half_adder(S, C, A, B);  
output S, C;  
input A, B;  
wire S, C, A, B;  
  
assign S = A ^ B;  
assign C = A & B;  
  
endmodule
```



```
module AndGate ( input wire a, input wire b, output wire y);  
  
    // Data flow modelling using continuous assignment  
  
    assign y = a & b;  
  
endmodule
```

In this example, the assign statement specifies that the output signal **y is equal to the logical AND (&) of input signals a and b.**

This relationship is continuously evaluated, reflecting the behavior of an AND gate.



Continuous assignments have the following characteristics.

- 1. The left hand side of an assignment must always be a scalar or vector net or a concatenation of scalar and vector nets. *It cannot be a scalar or vector register.*
- 2. Continuous assignments are always active. The assignment expression is evaluated as soon as one of the right-hand-side operands changes and the value is assigned to the left-hand-side net.
- 3. The operands on the right-hand side can be registers or nets or function calls. Registers or nets can be scalars or vectors.
- 4. Delay values can be specified for assignments in terms of time units. Delay values are used to control the time when a net is assigned the evaluated value. This feature is similar to specifying delays for gates. It is very useful in modeling timing behavior in real circuits.



Implicit Continuous Assignment

Instead of declaring a net and then writing a continuous assignment on the net, Verilog provides a shortcut by which a continuous assignment can be placed on a net when it is declared.

There can be only one implicit declaration assignment per net because a net is declared only once.

```
//Regular continuous assignment
wire out;
assign out = in1 & in2;

//Same effect is achieved by an implicit continuous assignment
wire out = in1 & in2;
```



Delays

Delay values control the time between the change in a right-hand-side operand and when the new value is assigned to the left-hand side.

Three ways of specifying delays in continuous assignment statements are

1. *Regular assignment Delay*
2. *Implicit continuous assignment delay*
3. *Net declaration delay.*



Regular assignment Delay

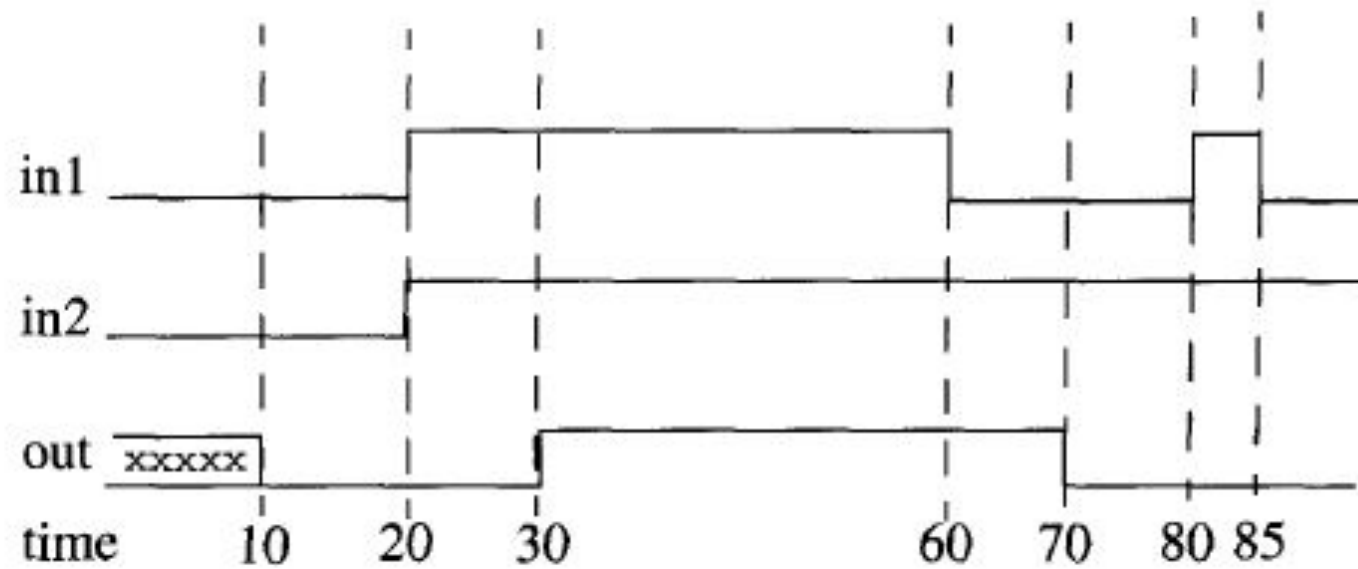
The delay value is specified after the keyword **assign**.

```
assign #10 out = in1 & in2; // Delay in a continuous assign
```

Any change in values of *in1* or *in2* will result in a delay of 10 time units before recomputation of the expression *in1* & *in2*, and the result will be assigned to *out*.

If *in1* or *in2* changes value again before 10 time units when the result propagates to *out*, the values of *in1* and *in2* at the time of recomputation are considered.

This property is called **inertial delay**. An input pulse that is shorter than the delay of the assignment statement does not propagate to the output.



1. When signals *in1* and *in2* go high at time 20, *out* goes to a high 10 time units later (time = 30).
2. When *in1* goes low at 60, *out* changes to low at 70.
3. However, *in1* changes to high at 80, but it goes down to low before 10 time units have elapsed.
4. Hence, at the time of recomputation, 10 units after time 80, *in1* is 0. Thus, *out* gets the value 0. A pulse of width less than the specified assignment delay is not propagated to the output.



Implicit continuous assignment delay

An equivalent method is to use an implicit continuous assignment to specify both a delay and an assignment on the net.

```
//implicit continuous assignment delay  
wire #10 out = in1 & in2;  
  
//same as  
wire out;  
assign #10 out = in1 & in2;
```



Net declaration delay.

A delay can be specified on a net when it is declared without putting a continuous assignment on the net.

If a delay is specified on a net *out*, then any value change applied to the net *out* is delayed accordingly.

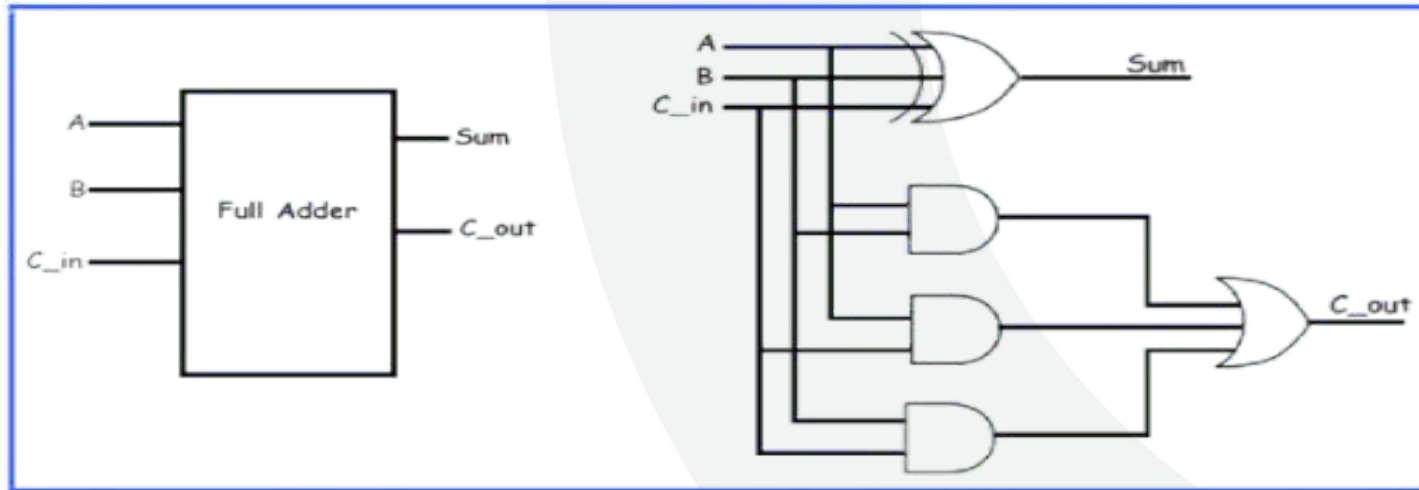
Net declaration delays can also be used in gate-level modeling.

```
//Net Delays
wire # 10 out;
assign out = in1 & in2;

//The above statement has the same effect as the following.
wire out;
assign #10 out = in1 & in2;
```

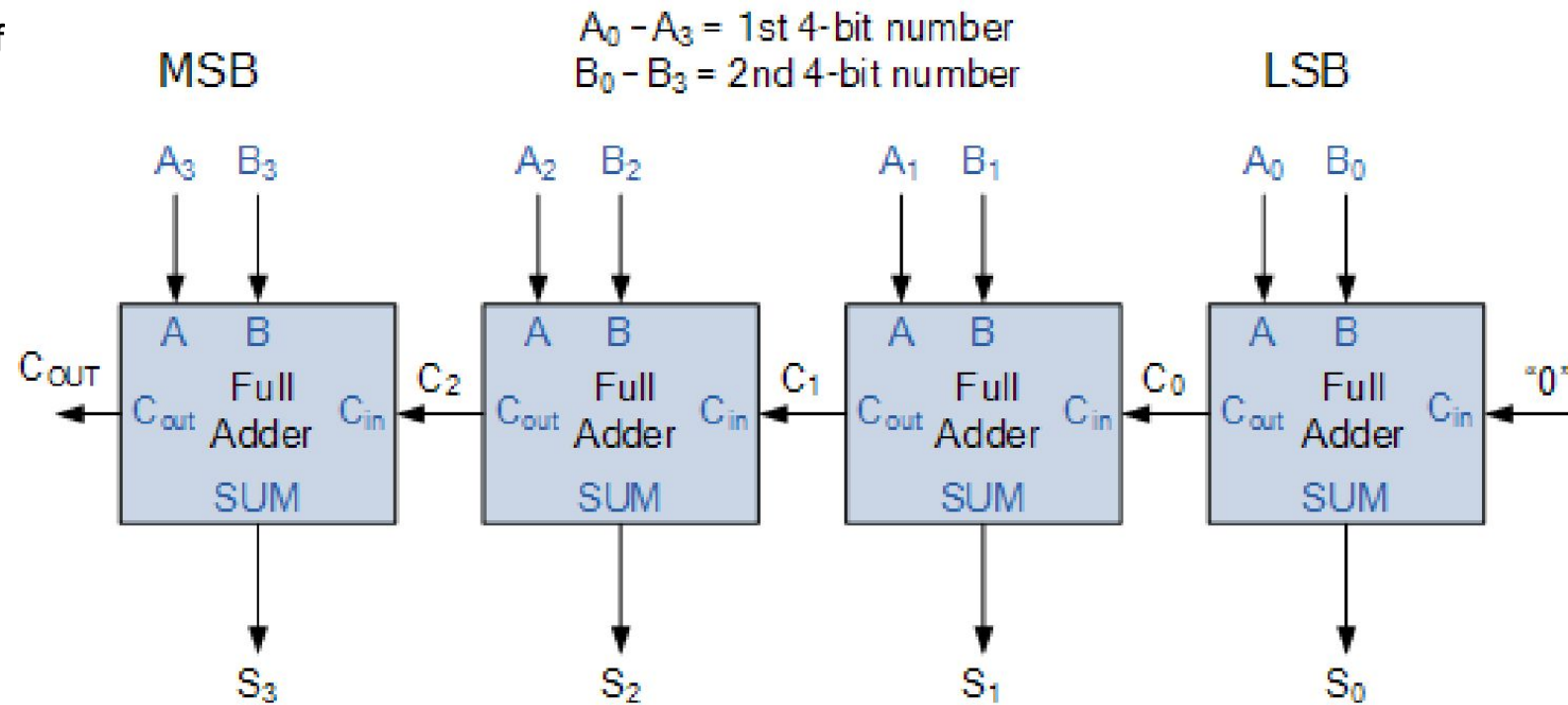
Dataflow model-Full adder

Go, change the world®



Example 3

```
module full_adder(Sum, C_out, A, B,Cin);  
output Sum, C_out;  
input A, B, Cin;  
  
assign Sum = A ^ B ^ Cin;  
assign C_out = (A & B) | (B & Cin) | (A & Cin);  
endmodule
```



```
module Four_bit_adder(input [3:0] A, input [3:0] B, output [3:0] Sum, output Carry );
```

```
assign { Carry, Sum }=A+B;
```

```
endmodule
```



Concatenation Operators

In Verilog, **concatenation operators** are used to combine multiple signals or variables into a **single vector**. The **concatenation operator** is **denoted by curly braces { }** and allows signals or expressions to be merged together.

Key Points:

- Signals are combined from left to right.
- **Operands must be sized.**(unsized operands are not allowed)
- The resultant signal will have a bit width equal to the sum of the bit widths of the individual signals.
- You can concatenate constants, wires, registers, part select, or outputs of expressions.



Ex 1: Simple Concatenation

```
wire [3:0] A = 4'b1010;  
wire [3:0] B = 4'b1100;  
wire [7:0] C;
```

```
// Concatenating A and B into an 8-bit signal C  
assign C = {A, B}; // C will be 8'b10101100
```

Ex 3: Concatenating Constants and Signals

```
wire [3:0] G = 4'b1001;  
wire [7:0] H;
```

```
// Concatenating a constant and signal  
assign H = {4'b0000, G}; // H will be 8'b00001001
```

Ex 2: Concatenating Different Sized Signals

```
wire [1:0] D = 2'b01;  
wire [3:0] E = 4'b1111;  
wire [5:0] F;
```

```
// Concatenating D and E into F (6-bit signal)  
assign F = {D, E}; // F will be 6'b011111
```

Ex 4: Replication using Concatenation

```
wire [1:0] J = 2'b11;  
wire [7:0] K;
```

```
// Replicating J four times  
assign K = {4{J}}; // K will be 8'b11111111
```



Use Cases of Concatenation in Digital Design

1. **Bit Manipulation:** Concatenation is used to combine smaller vectors into larger ones for operations like shifting, combining flags, or packing data.
2. **Zero Padding:** Useful for padding smaller width signals to match the required width of larger buses.
3. **Bus Construction:** Constructing larger data buses or control signals from multiple smaller components.
4. **Replication:** Repeating bit patterns for data extension or signaling purposes.



1) Design a 2:1 MUX using CAS in Verilog

```
Module mux2to1 (i, s, y);
```

```
input [1:0] i;
```

```
input s;
```

```
output y;
```

```
assign y = (i[0] & (~s)) | (i[1] & (s));
```

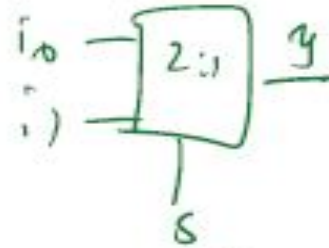
or

```
assign y = s ? i[1] : i[0];
```

or

```
assign y = (s == 1) ? i[1] : i[0];
```

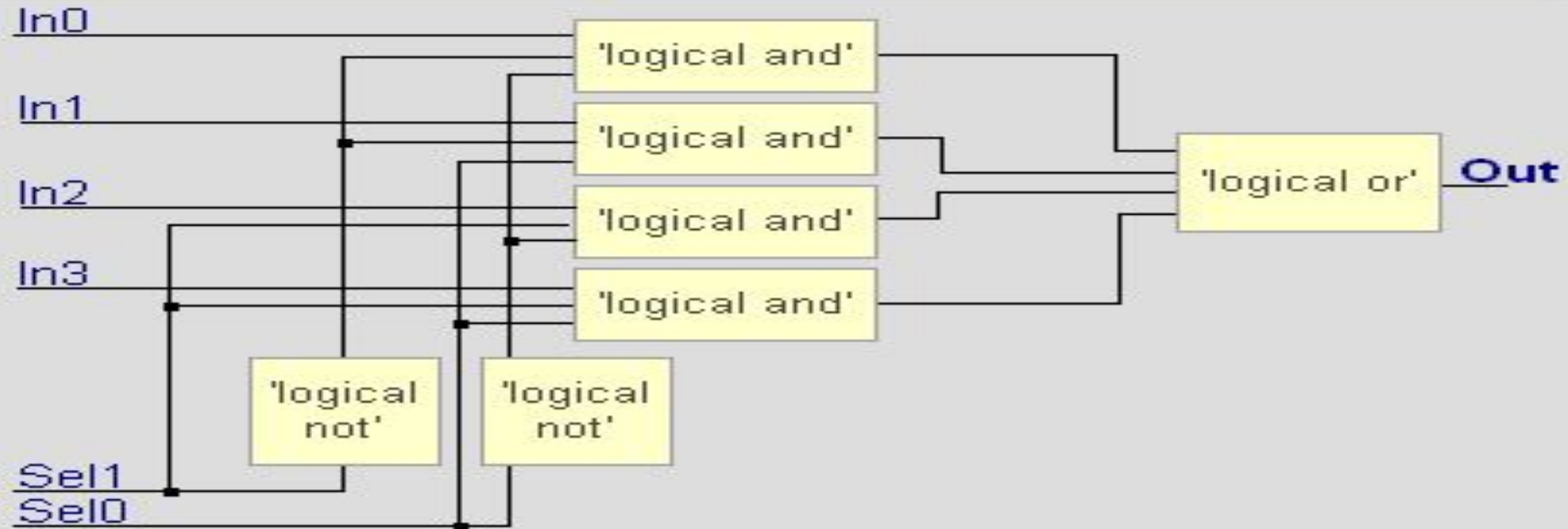
```
Endmodule
```





Data flow model- MUX 4:1

Go, change the world®



```
module mux_4_to_1 (Out, In0, In1, In2, In3, Sel1, Sel0);

output Out;
input In0, In1, In2, In3, Sel0, Sel1;

assign Out = (~Sel1 & ~Sel0 & In0) | (~Sel1 & Sel0 & In1)
              | (Sel1 & ~Sel0 & In2) | (Sel1 & Sel0 & In3);

endmodule
```

2) Design a 4:1 Multiplexer using assignment & conditional operator.

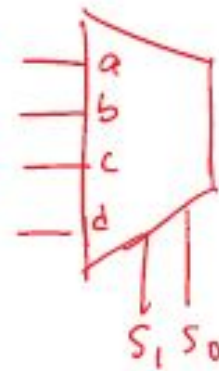
```
module mux4to1(a,b,c,d,s1,s0,y);
```

```
input a,b,c,d;
```

```
output y;
```

```
input s1,s0;
```

```
assign y = ((~s0) & (~s1) & a) | ((~s0) & (s1) & b) |  
            ((s0) & (~s1) & c) | ((s0) & (s1) & d);
```



s ₁	s ₀	y
0	0	a
0	1	b
1	0	c
1	1	d

with Conditional Operator:-

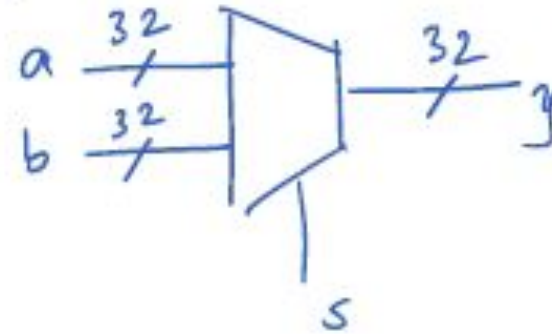
```
assign y = s1 ? s0 ? d : c : s0 ? b : a;
```

```
Endmodule
```

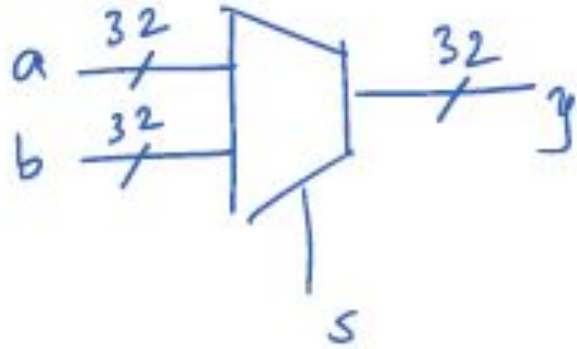


RV College of
Engineering®

3) Write a Verilog code for following design



3) Write a Verilog code for following design



module design1 (a, b, s, y)

input [31:0] a, b; ✓

output [31:0] y ✓

input s;

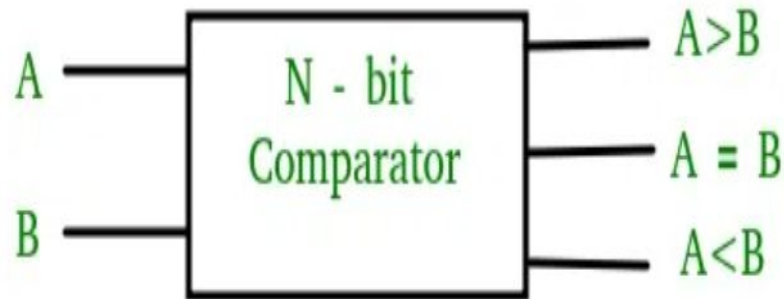
assign y = s ? a : b;

Endmodule



Magnitude Comparator in Digital Logic

A magnitude digital Comparator is a combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than, or greater than the other binary number.



A	B	A < B	A = B	A > B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

$$A > B: AB'$$

$$A < B: A'B$$

$$A = B: A'B' + AB$$



Magnitude Comparator in Digital Logic

A	B	A<B	A=B	A>B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

$A > B: AB'$

$A < B: A'B$

$A = B: A'B' + AB$

```
module comparator_1bit ( a ,b ,equal ,greater ,lower );
```

```
output equal ;  
output greater ;  
output lower ;
```

```
input a ;  
input b ;
```

```
assign equal = a ~^ b;  
assign lower = (~a) & b;  
assign greater = a & (~b);
```

```
endmodule
```



2 bit Magnitude Comparator in Digital Logic

A1	A0	B1	B0	A_greater_ B	A_equal_B	A_less_B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0



RV College of
Engineering

4) Write a Verilog code for 2-bit Magnitude comparator
Using CAS { Home work }

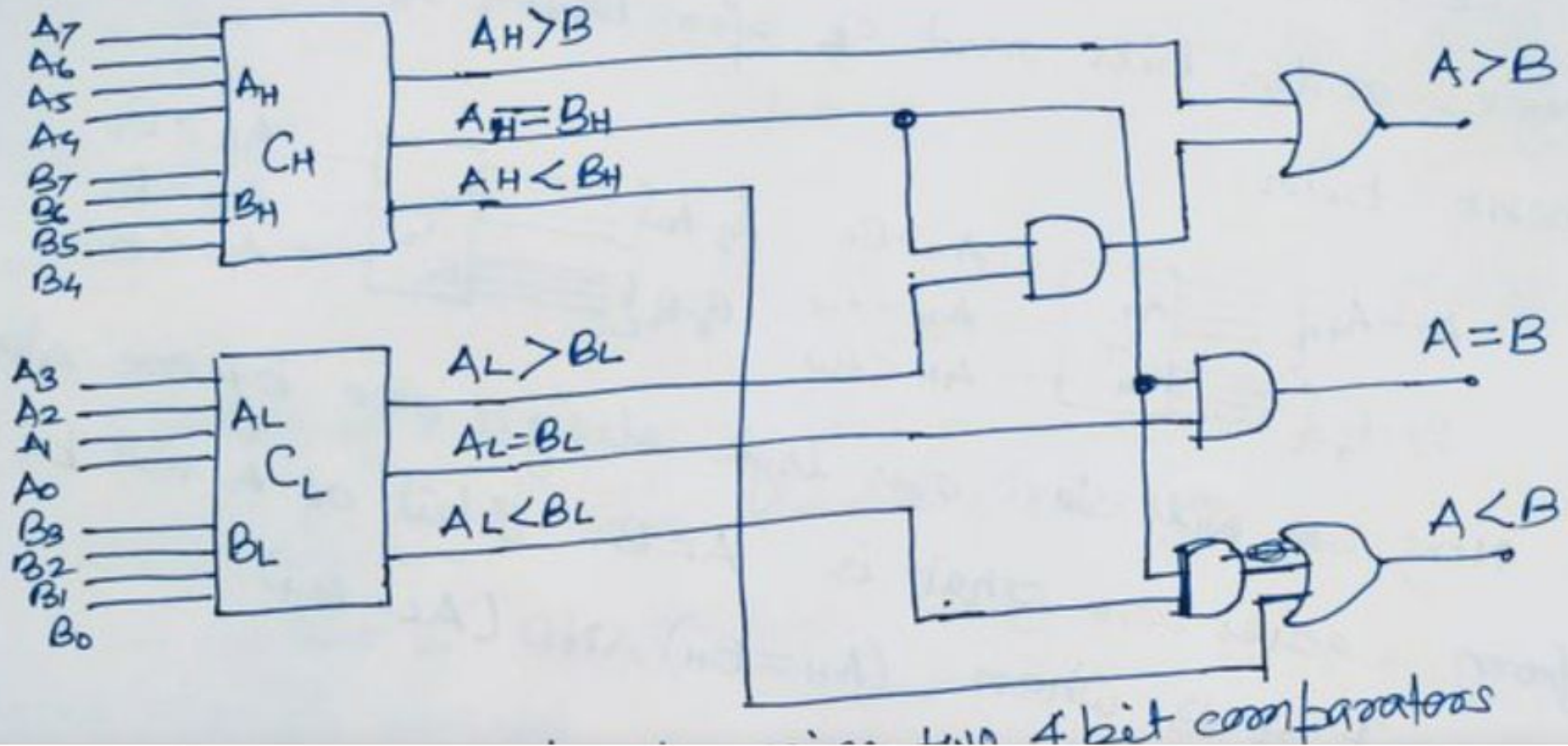
The specification of the 2-bit comparator is as follows:

- Input: 2-bit A and B for comparison
- Output:
 - A_greater_B: high if $A > B$ else low
 - A_equal_B: high if $A = B$ else low
 - A_less_B: high if $A < B$ else low

5) Write a Verilog code that produces a 4-bit output
indicating number of 1's in a 8 bit no. word.



```
module two_bit_comparator_df(  
input [1:0] A, // 2-bit input A  
input [1:0] B, // 2-bit input B  
output reg A_gt_B, // Output: A is greater than B  
output reg A_eq_B, // Output: A is equal to B  
output reg A_lt_B // Output: A is less than B);  
  
// Continuous assignment statements using conditional operators  
assign A_gt_B = (A > B) ? 1'b1 : 1'b0;  
assign A_eq_B = (A == B) ? 1'b1 : 1'b0;  
assign A_lt_B = (A < B) ? 1'b1 : 1'b0;  
endmodule
```





RV College of
Engineering®

WRITING A SIMPLE TESTBENCH

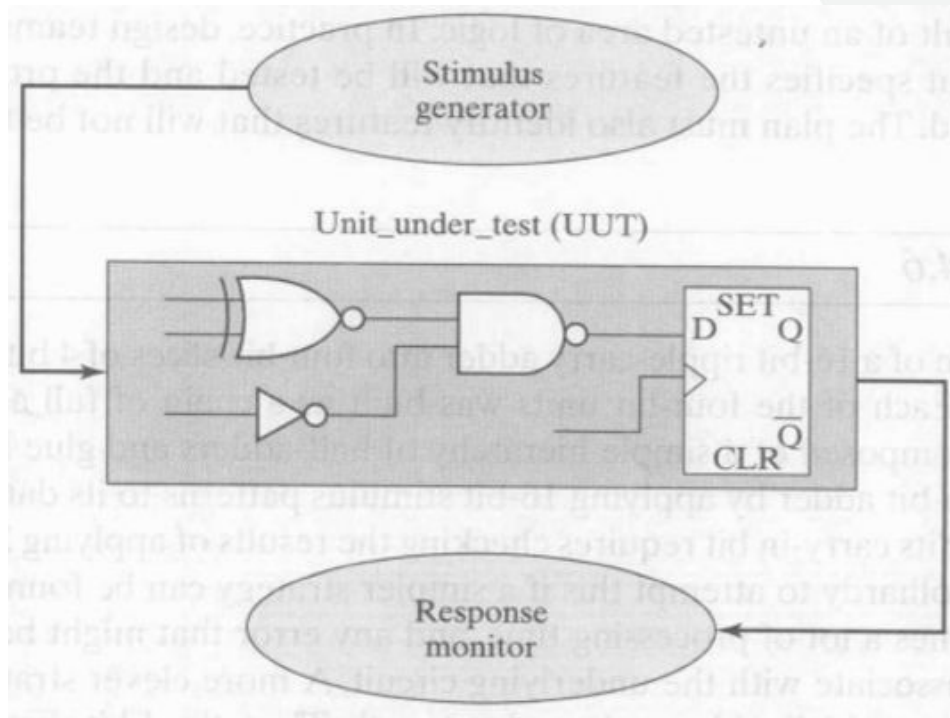
Go, change the world®

A testbench is an HDL program that describes and applies a stimulus to an HDL model of a circuit to test it and to observe its response during simulation.

Testing methodology

Go, change the world®

A logic circuit must be tested systematically to ensure that all of its logic has been exercised and functionally correct



Test bench: applies stimulus patterns to the circuit and displays output waveforms

Separate verilog module which resides at the top of the design

Testing environment consists of stimulus generator, response monitor and an instantiation of the unit under test



Testing methodology - cntd

Stimulus generator: verilog statement to define patterns

Response monitor: selectively gathers data on signals within the design and displays in text or graphical format

Simulator task:

- (1) checks source code,
- (2) reports any violations of the language's syntax
- (3) determines whether the number of ports specified by a model are actually connected to it in an instantiation
- (4) simulates the behavior of the circuit under the application of input signals that are defined in testbench

Syntax errors must be eliminated before a simulation can run



Signal generators for testbench

Go, change the world®

- Initial declares a single pass behavior that begins executing when the simulator is activated at `tsim=0`
- Statements within `begin` and `end` are called procedural statements
- `=` operator is procedural assignment
- Statements executes sequentially from top to bottom
- `#` is a delay control operator: suspends the execution of that statement and hence subsequent statement until specified time

```
module halfadd;  
reg a,b;  
wire carry,sum;  
halfadd    u1(carry, sum,  
a ,b);  
Initial begin  
#100 $finish;  
end
```

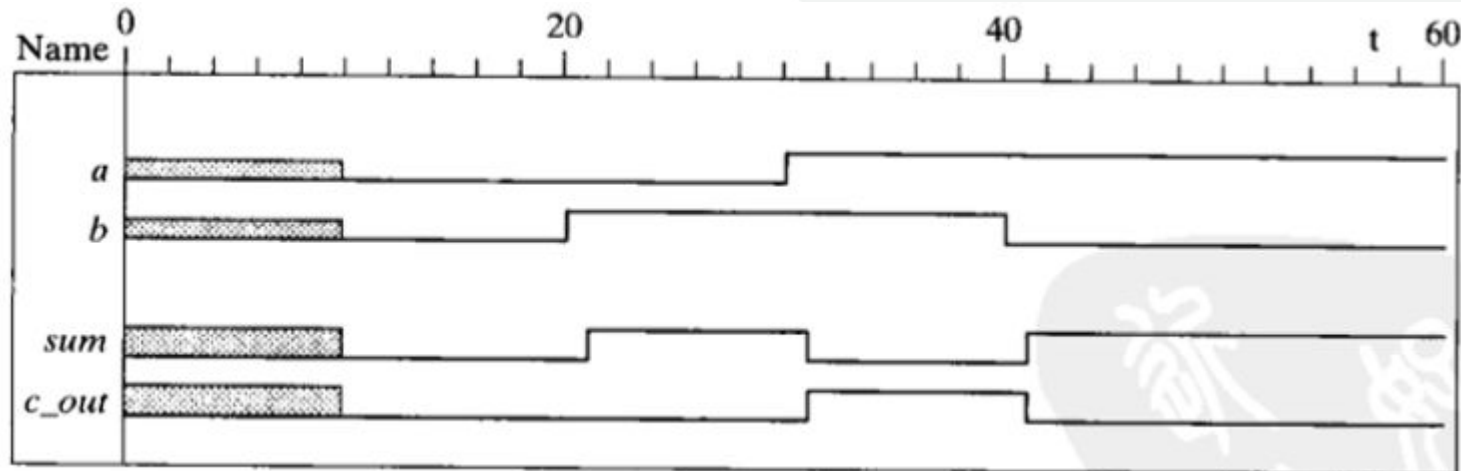
```
Initial  
begin  
#10 a=0'b0;b=0'b1;  
#10 a=0'b1;b=1'b0  
#10 a=1'b0;b=0'b0 ;  
#10 a=1'b1;b=1'b1;  
end  
endmodule
```



Testbench

Go, change the world®

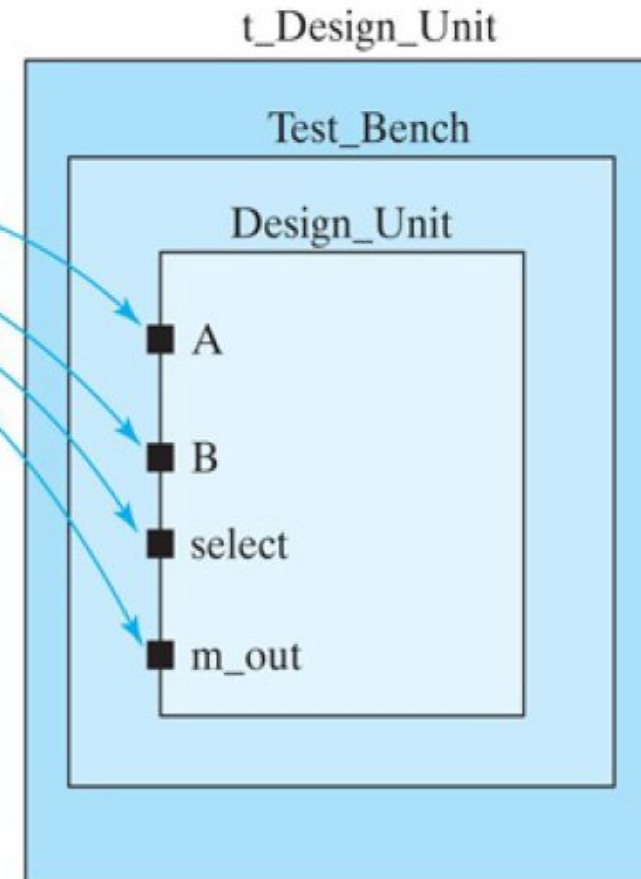
- Reg variable takes the value x in the beginning of simulation until it is loaded with different values by explicit statement
- Each statement is delayed by 10 unit
- Delay accumulates (40 u)
- Simulation terminates after 100 u





Interaction between testbench and Verilog design unit

```
module t_Design_Unit ();  
// Declare stimulus signals  
reg t_A,t_B,t_select;  
//Declare reponse signals  
wire : m_out;  
--Instantiate unit under test  
Design_Unit UUT  
(.A (t_A),.B(t_B),.select(t_select)t,.m_out(t_m_out));  
  
--Generate stimulus  
initial begin  
t_a <= '1';  
t_B <= '0';  
end;  
  
initial begin  
select = 1; forever # 10 select = ! select;  
end;  
end;  
endmodule
```





Test Bench-Full adder

Go, change the world®

```
module full_adder(Sum, C_out, A, B,Cin);  
output Sum, C_out;  
input A, B, Cin;  
  
assign Sum = A ^ B ^ Cin;  
assign C_out = (A & B) | (B & Cin) | (A & Cin);  
endmodule
```

```
module full_adder_tb;  
reg a,b,cin;  
wire sum,carry;
```

```
full_adder DUT(sum, carry,a, b ,cin);
```

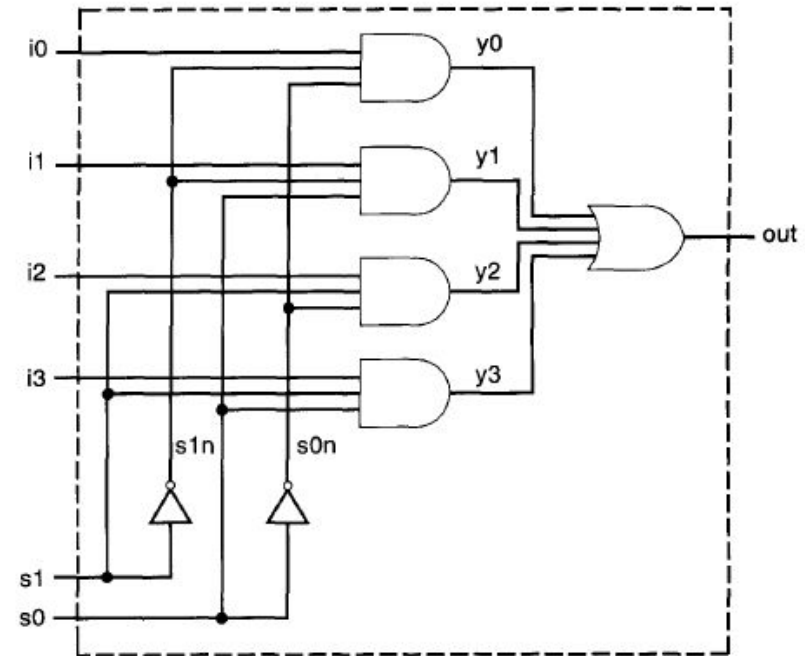
```
initial begin  
a = 1; b = 0; cin = 0;  
#10  
a = 0; b = 0; cin = 1;  
#10  
a = 0; b = 1; cin = 0;  
#10  
a = 0; b = 1; cin = 1;  
#10  
a = 1; b = 1; cin = 1;  
#10  
$finish();  
end
```

```
endmodule
```



Verilog Description of Multiplexer(4:1)

```
module mux4-to-1 (out, i0, i1, i2, i3, s1, s0);  
    output out;  
    input i0, i1, i2, i3;  
    input s1, s0;  
  
    // Internal wire declarations  
    wire sln, son;  
    wire y0, y1, y2, y3;  
  
    // Gate instantiations  
    // Create sln and son signals.  
    not (sln, s1) ;  
    not (son, s0);  
    // 3-input and gates instantiated  
    and (y0, i0, sln, son);  
    and (y1, i1, sln, son);  
    and (y2, i2, s1, son);  
    and (y3, i3, s1, s0);  
    // 4-input or gate instantiated  
    or (out, y0, y1, y2, y3);  
endmodule
```





TestBench: *Multiplexer(4:1)*

```
// Define the stimulus module (no ports)

module stimulus;
// Declare variables to be connected to inputs
reg IN0, IN1, IN2, IN3;
reg S1, S0;

// Declare output wire
wire OUTPUT;

// Instantiate the multiplexer
mux4-to-1 mymux (OUTPUT, IN0, IN1, IN2, IN3, S1, S0);
```



RV College of
Engineering®

```
initial
begin
// set input lines
IN0 = 1; IN1 = 0; IN2 = 1; IN3 = 0;

#1 $display("INO= %b, IN1= %b, IN2= %b, IN3=%b\n", IN0, IN1, IN2, IN3);

// choose IN0
S1 = 0; so = 0;
#1 $display("S1 = %b, SO = %b, OUTPUT = %b \n", S1, SO, OUTPUT);

// choose IN1
S1 = 0; so = 1;
#1 $display("S1 = %b, SO = %b, OUTPUT = %b \n", S1, SO, OUTPUT);

// choose IN2
S1 = 1; SO = 0;
#1 $display("S1 = %b, SO = %b, OUTPUT = %b \n", S1, SO, OUTPUT);
end

endmodule
```